



Software Engineering Department

Capstone Project Phase B – 61998

Modeling Citation Trustworthiness in Evolving Scholarly Networks

26-1-R-9

Fatma Dabbah	213675549	<u>fatma.dabbah@e.braude.ac.il</u>
Manal Nama	325393999	<u>manal.nama@e.braude.ac.il</u>

Supervisors:

Dr. Dvora Toledano-Kitai

Prof. Zeev Volkovich

GitHub Link: <https://github.com/Manal2308/Final-Project.git>

Table of Contents

Abstract	2
1. Introduction and Problem Formulation	3
2. Theoretical Background and Preliminaries	5
2.1 Citation Networks	5
2.2 Embedding	5
2.3 Graph Embedding Methods	6
2.3.1 Node2Vec	6
2.3.2 GraphSAGE	6
3. Research Process and System Implementation	7
3.1.1 General Description	7
3.1.2 General Implementation Idea	7
3.1.3 Target Users	8
3.1.4 Model Architecture	8
3.2 Research Process	9
3.2.1 Solution Description and General Overview	10
3.2.2 Stage 0: Data Preparation and Snapshot Construction	10
3.2.3 Stage 1: Feature Extraction	10
3.2.4 Stage 2: Noise Injection and Perturbation	10
3.2.5 Stage 3: Graph Embedding Methods	11
3.2.6 Stage 4: Link Prediction and Reconstruction	11
3.2.7 Stage 5: Stability Computation and Evaluation	11
3.2.8 Stage 6: Result Saving and Checkpoint Mechanism	12
3.2.9 Testing and Validation	12
3.3 Tools and Technologies	12
3.3.1 Programming Language	12
3.3.2 Development Environment	12
3.3.3 Dataset	13
3.3.4 Main Libraries and Frameworks	13
3.3.5 Graph Embedding and Machine Learning Libraries	14
3.3.6 Experiment Management and Storage Tools	14
3.4 Analytical and Engineering Challenges During Implementation	14
3.4.1 Large Dataset Size	14
3.4.2 RAM Limitations	15
3.4.3 GPU Limitations	15
3.4.4 Long Execution Time	15
3.4.5 Lack of Ground-Truth Labels	15
3.4.6 Limitations of the Proposed Solution	16
4. Results, Conclusions, and Project Evaluation	16
4.1 Results Visualization	16
4.2 Conclusions	20

4.3 Project Goals Evaluation	20
4.4 Decision Making and Handling Challenges	21
4.5 Lessons Learned	21
4.6 Future Work	21
References	22
Appendices:	23
A. User Guide	23
B. Maintenance Guide	28

Abstract

This project presents a graph-based framework for assessing citation trustworthiness in evolving scholarly citation networks. Using data from OpenAlex, the system builds cumulative yearly citation graph snapshots, injects artificial noise edges, removes sampled edges, and evaluates whether graph embedding models can reconstruct them. The main assumption is that reliable citation links exhibit stronger structural support and therefore higher stability than artificial or weak links.

The project compares two graph embedding methods: Node2Vec and GraphSAGE. The results show that original citation edges are generally more stable than injected noise edges, and that GraphSAGE provides stronger and more consistent separation than Node2Vec. The evaluation is based on stability scores, AUC-ROC, coefficient of variation, distribution-based analysis, and node-level anomaly analysis. Overall, the framework provides structural indicators for identifying potentially weak or suspicious citation links that may require further investigation.

Keywords: Dynamic Citation Networks, Graph Embeddings, Network Perturbations, Temporal Stability, Anomaly Detection.

1. Introduction and Problem Formulation

The academic citation system plays a central role in evaluating scientific influence, connecting related studies, and measuring the contribution of researchers, papers, and journals. Citations are used not only as references to previous work, but also as indicators of academic impact and scientific relevance [7]. However, in recent years, the reliability of this system has been increasingly challenged by citation manipulation, including artificial citations, excessive self-citation, forced citations, and citation cartels [3,5,6,11]. These practices may distort academic impact indicators because they create citation links that do not necessarily represent genuine scientific relevance or meaningful scholarly contribution.

The main problem addressed in this project is how to assess the trustworthiness of individual citation links within a large and evolving citation network, especially when there are no explicit ground-truth labels indicating which citations are genuine and which are suspicious. Traditional approaches based mainly on citation counts, journal impact metrics, or manual inspection are insufficient for large-scale scholarly networks. Therefore, this project treats each citation as a structural link in a graph and examines whether this link is supported by the broader topology of the citation network.

The main objective of the project is to develop and implement a computational framework for evaluating citation trustworthiness using graph-based methods. The framework aims to distinguish between structurally supported citation links and artificial or topologically unsupported links. In addition, the project examines how citation stability changes over time by analyzing cumulative yearly snapshots of the citation network [4].

The approach proposed in Phase A is based on the assumption that a reliable citation is expected to be part of a stable topological structure. In other words, if such a citation link is temporarily removed from the graph, the surrounding network structure should still provide enough indirect evidence to reconstruct it. In contrast, artificial or topologically unsupported citation links are expected to have lower structural support, making them harder to reconstruct after removal.

To evaluate this idea, the project uses controlled graph perturbation. In this process, selected citation edges are removed from the graph, node embeddings are generated, and a link prediction task is applied to estimate whether the removed edges can be reconstructed [1,2]. Edges that are consistently reconstructed across repeated perturbation runs are considered structurally stable, while edges that are rarely reconstructed are considered potentially suspicious.

The implementation relies on several key concepts introduced in Phase A. A citation network is represented as a graph in which papers are nodes and citation relationships are edges. In the implementation, the citation graph is processed as an undirected

graph in order to focus on structural relationships between papers rather than citation direction.

Graph embedding methods transform graph nodes into numerical vectors that preserve structural information. Link prediction is then used to estimate the likelihood of a connection between two nodes based on their embeddings [10]. Two embedding methods were discussed: Node2Vec [8], which uses biased random walks and is mainly suitable for static graph analysis, and GraphSAGE [9], an inductive method that learns node representations by aggregating information from neighboring nodes and is therefore more suitable for dynamic and evolving graphs.

Phase A also introduced the temporal aspect of the problem. Citation networks are not fixed; they grow over time as new papers and citation links are added. Accordingly, the project analyzes citation stability across cumulative temporal snapshots rather than at a single time point [4].

Each snapshot represents the citation network up to a specific year, and the reconstruction process is applied independently to each snapshot in order to compute stability scores over time.

In Phase B, the project continues from the theoretical foundation defined in Phase A and focuses on the practical implementation of the proposed framework. The work includes constructing yearly snapshots, injecting artificial noise edges, applying graph embedding models, performing edge reconstruction, computing stability scores, saving experimental results, and comparing the behavior of original citation edges with injected artificial edges. The main goal of this phase is to evaluate whether the implemented model can distinguish between structurally supported citation links and artificial citation links that lack stable topological support.

The contribution of this project is the implementation of an experimental pipeline for studying citation trustworthiness in evolving scholarly networks. The pipeline combines temporal citation graph construction, artificial noise injection, graph embedding methods, and link prediction-based reconstruction into one framework.

This enables a systematic comparison between original citation links and injected artificial links, and provides a basis for analyzing structural stability as an indicator of citation reliability.

The remainder of this book is organized as follows.

Section 2 presents a short theoretical background and preliminaries, including the main concepts required to understand the project, such as citation networks, graph embeddings, Node2Vec, GraphSAGE, link prediction, and stability analysis.

Section 3 describes the research process, the proposed model, and the implementation. This section includes the model objectives, the target users and usage scenario, the general structure of the implemented solution, the main system

components, and the research development process. It also presents the tools, technologies, programming environment, libraries, and datasets used in the project, as well as the main challenges and limitations encountered during the implementation.

Section 4 presents the results and conclusions. It describes the experiments, evaluation metrics, main results, graphs, and analysis of the obtained findings. Finally, the section summarizes the main conclusions, discusses the contribution of the project, and suggests possible directions for future work.

2.Theoretical Background and Preliminaries

2.1 Citation Networks

A citation network is a formal mathematical model that represents the complex relationships of “who cites whom” within a collection of publications, such as academic articles [7]. A citation network is represented as a graphical structure that allows for in-depth analysis of knowledge flow, research influence, the formation of subfields, and the identification of unusual patterns and unnatural behavior in the academic system [4].

In this network, each node represents a single publication, and an edge $A \rightarrow B$ indicates that article A cited article B . Formally, the network is defined as a directed graph $G = (V, E)$, where V represents the collection of publications and E the set of citation links. An edge $(u, v) \in E$ exists if and only if publication u cites publication v .

The presence of a citation does not necessarily indicate quality or genuine scientific contribution. Articles may cite each other for a variety of reasons, such as criticism, strategic reasons, or due to phenomena like self-citation, forced citation, or citation cartels [3,5,6,11]. Citation network analysis often combines statistical tools, structural analysis, chronological examination, and sometimes textual context analysis to identify unusual patterns.

2.2 Embedding

Embedding is a key technique in machine learning that allows mapping discrete objects (such as text, images, audio, or graph nodes) into a continuous, relatively low-dimensional vector space R^d .

While many machine learning algorithms are limited to accepting only numerical input, raw data (such as words or network connections) does not come in this format. Embedding bridges this gap by converting objects into information-rich vectors.

The location of each vector captures semantic or structural relationships, so similar objects are positioned close to each other in the space.

In citation networks, embedding allows nodes to be represented numerically while preserving structural or semantic similarity: nodes with similar roles or context in the network appear close in the embedding space. Similarity between nodes is typically measured via cosine similarity or Euclidean distance.

2.3 Graph Embedding Methods

This project uses two complementary graph embedding methods:

2.3.1 Node2Vec

Node2Vec [8] is a transductive method based on biased random walks and the skip-gram model. The method converts the graph into sequences of nodes, similar to sentences in text, and learns the vector representation using a skip-gram model. Hyperparameters p (return) and q (in-out) control the balance between local and global exploration. Node2Vec is mainly suitable for static networks.

2.3.2 GraphSAGE

GraphSAGE [9] is an inductive embedding algorithm for learning on graphs. Unlike Node2Vec, which learns a separate vector representation for each existing node, GraphSAGE learns a general function that generates embeddings for both existing and new nodes, based on the node's properties and the local structure of its neighbors in the graph.

The algorithm works by sampling a fixed number of neighbors for each node and aggregating their information (e.g., mean, LSTM, or pooling), which is combined with the node's representation to produce an embedding capturing both node features and structural context.

This process can be performed in multiple layers, with each layer expanding the node's field of view to more distant neighbors.

This approach makes GraphSAGE scalable and particularly suitable for dynamic graphs.

These methods capture both local neighborhood structure and broader network topology. This approach builds on prior work in anomaly detection and link prediction in dynamic networks, leveraging structural patterns to identify unstable or unusual connections [1,2].

2.4 Link Prediction

Link prediction [10] is a central task in network analysis, aimed at estimating the likelihood of a connection between two nodes that are not directly linked in the network. In citation networks, it predicts whether a genuine citation relationship exists between two papers, even if the corresponding edge is missing or temporarily

removed. This approach is based on the assumption that the network’s topological structure, together with indirect connections, contains sufficient information to infer latent links. High similarity between node embeddings indicates a high likelihood of a connection.

In this project, link prediction is used to evaluate citation stability: reliably supported citations can be consistently reconstructed, whereas manipulative or structurally weak citations tend to be less predictable and exhibit lower reconstruction stability. Metrics such as AUC-ROC, Detection Rate, and Distribution Distance are used to quantify the separation between genuine and artificial citation patterns, providing a principled measure of the model’s discrimination capability.

3. Research Process and System Implementation

This chapter describes the proposed research approach and the implementation process of the project. It presents the general structure of the developed framework, the main components of the model, and the research pipeline used to evaluate citation trustworthiness in evolving scholarly networks.

3.1 Proposed Approach

This section presents the general idea of the proposed solution, including the purpose of the model and its overall architecture.

3.1.1 General Description

The system developed in this project is a research-oriented computational framework designed to analyze the trustworthiness of citation links in evolving scholarly networks. The main goal of the system is to examine whether citation relationships are structurally supported by the citation network or whether they behave like weak, unstable, or artificially injected links.

Since the project is research-oriented, the system is not intended to serve as a commercial software product, but rather as an experimental pipeline for evaluating citation reliability using graph-based machine learning methods.

3.1.2 General Implementation Idea

The system is based on the assumption that genuine citation links are usually supported by the surrounding graph structure. Therefore, if a reliable citation edge is temporarily removed from the graph, the model should still be able to reconstruct it using indirect structural information from the remaining network. In contrast, artificial or structurally weak citation links are expected to receive lower reconstruction scores, since they do not naturally fit the topology of the citation network [1,2].

To implement this idea, the system is organized as a modular research pipeline. The pipeline receives citation data as input, constructs temporal graph representations,

applies graph-based learning methods, and produces stability-based results for citation links. This modular structure allows the experiment to be executed gradually, tested more easily, and extended in future work.

3.1.3 Target Users

The main users of the system are researchers, students, and academic supervisors who are interested in citation network analysis, graph-based machine learning, anomaly detection, and citation manipulation detection. Since the project is research-oriented, the system is not intended for general end-users, but rather for users with a technical or academic background who need to run experiments and analyze citation network behavior.

A typical usage scenario begins with a citation dataset containing papers, publication years, and citation links. The system processes the dataset, builds yearly citation graph snapshots, injects artificial noise edges, applies embedding-based reconstruction, and outputs stability scores and evaluation summaries. These results can then be used by researchers to study whether structurally unsupported citation links can be distinguished from genuine citation links.

3.1.4 Model Architecture

The architecture of the proposed model is based on two complementary processes: structural stability analysis and temporal stability analysis [1,2].

The Structural Stability Model, shown in Figure 1, describes the process applied to a single citation graph snapshot. In this process, citation edges are temporarily removed, node embeddings are generated using Node2Vec [8] or GraphSAGE [9], and a link prediction task is used to evaluate whether the removed edges can be reconstructed [10]. The reconstruction results are accumulated across iterations and used to compute a stability score for each evaluated edge.

The Temporal Stability Analysis, shown in Figure 2, extends the structural model across cumulative yearly snapshots. For each snapshot, the structural stability process is applied independently, and the obtained stability scores are stored and aggregated over time. This enables the system to analyze how citation stability changes as the citation network evolves.

Together, these two processes define the overall architecture of the implemented research pipeline. This architecture supports the comparison between original citation edges and injected artificial noise edges, and enables the evaluation of citation trustworthiness from both structural and temporal perspectives.

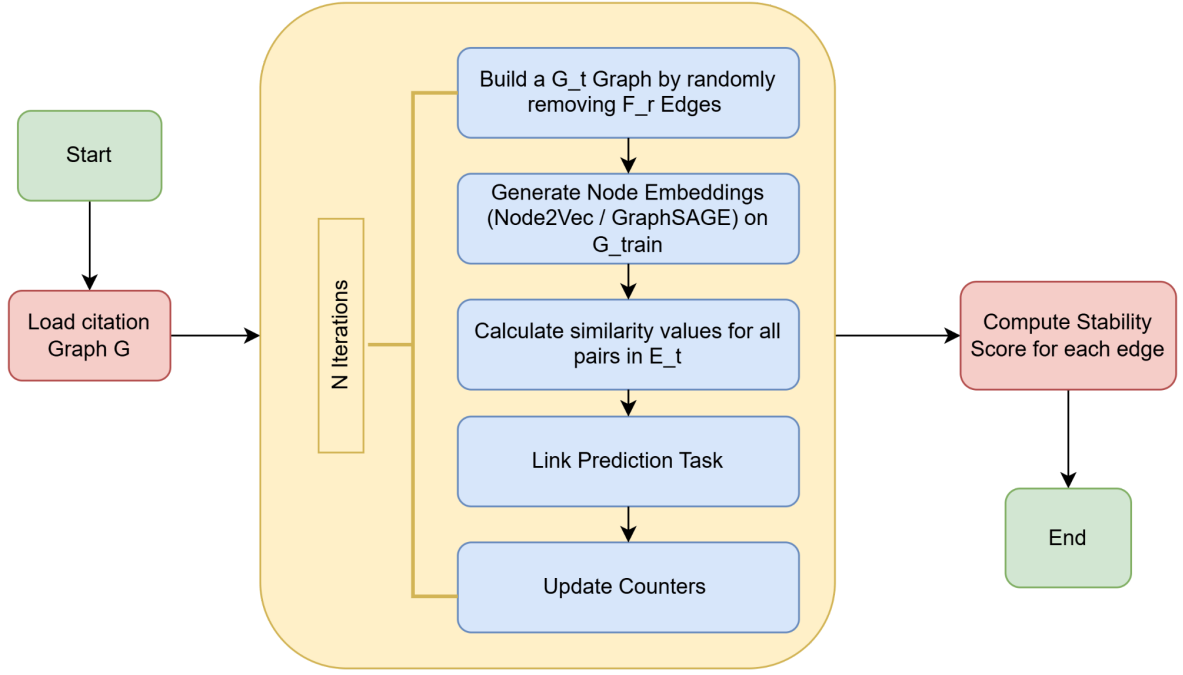


Figure 1. Flowchart of the Structural Stability Model.

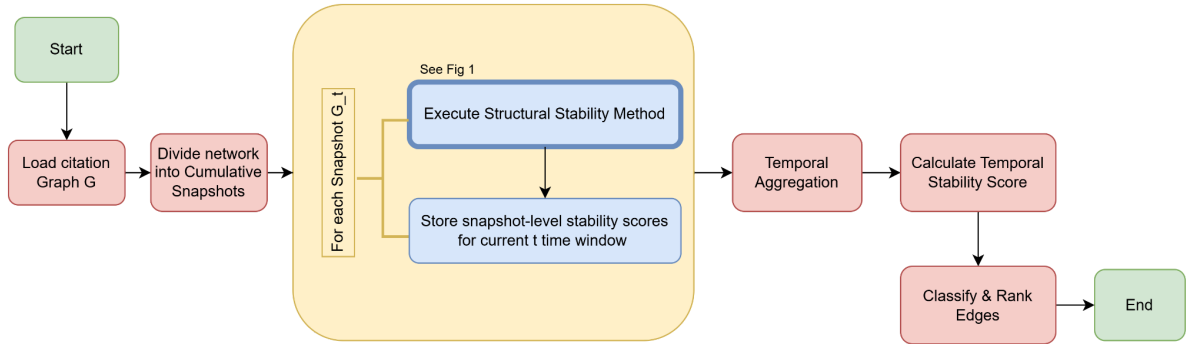


Figure 2. A flowchart of the Temporal Stability Analysis.

3.2 Research Process

This section describes the main research process followed during the project implementation. The process includes preparing the citation dataset, constructing yearly snapshots, extracting node features, injecting artificial noise edges, generating graph embeddings, performing link prediction, computing stability scores, and saving the results.

3.2.1 Solution Description and General Overview

The system is not a commercial product or a final decision-making tool. Instead, it provides stability-based indicators for citation links that may require further investigation.

The solution receives citation data as input and transforms it into yearly cumulative graph snapshots. Each snapshot includes the papers and citation links that exist up to a specific year. This allows the system to analyze the citation network not only as a static structure, but also as a dynamic network that grows over time [4].

3.2.2 Stage 0: Data Preparation and Snapshot Construction

In this stage, the citation dataset is prepared for graph-based analysis. The input data consists of academic papers and citation relationships, where each paper is represented as a node and each citation is represented as an edge.

The dataset is first cleaned and filtered by publication year. Invalid citation or inconsistent links are removed, and isolated nodes may be excluded when appropriate. Following preprocessing, the system constructs cumulative yearly snapshots of the citation network, where each snapshot includes all papers and citation links up to a given year.

For each yearly snapshot, an undirected graph is constructed. Although citation relationships are inherently directed, this project converts them into an undirected representation to focus on the existence and strength of structural connections between papers, rather than on citation directionality. This design choice supports downstream tasks such as graph embedding and link prediction.

3.2.3 Stage 1: Feature Extraction

The solution includes a feature extraction stage in which each paper is represented by numerical features from metadata and graph structure. These features include publication year, number of references, number of citations, internal reference count, internal citation count, PageRank, degree centrality, and clustering coefficient.

These features are mainly used by the GraphSAGE model, since GraphSAGE learns node representations by combining node attributes with neighborhood information [9].

3.2.4 Stage 2: Noise Injection and Perturbation

To evaluate the ability of the model to distinguish between genuine and artificial links, the system injects artificial noise edges into the graph. Artificial edges are added at a rate of 5% of the original citation edges. These edges simulate citation links that do not necessarily have real structural support in the citation network [1,2].

After injecting the noise edges, the system performs a perturbation process. In this step, part of the edges is temporarily removed from the graph, and the model attempts to reconstruct them using the learned node representations [1,2].

To reduce RAM consumption, the implementation uses an edge-sampling strategy. In each iteration, 20% of the original citation edges are selected as a representative sample, and 20% of them are temporarily removed for reconstruction.

This strategy reduces memory usage while preserving the main goal of the experiment: comparing the reconstruction behavior of original citation edges and injected artificial noise edges.

3.2.5 Stage 3: Graph Embedding Methods

The system uses two graph embedding algorithms: Node2Vec [8] and GraphSAGE [9].

Node2Vec learns node embeddings using biased random walks over the graph. This allows the model to capture structural relationships between papers based on their positions and neighborhoods in the graph [8].

GraphSAGE learns node representations by aggregating information from neighboring nodes and node features. This makes it suitable for dynamic and evolving graph settings, where new nodes and edges may appear over time [9].

Using both methods allows the project to compare two different embedding approaches and evaluate their effectiveness in citation link reconstruction.

3.2.6 Stage 4: Link Prediction and Reconstruction

After generating the node embeddings, the system performs a link prediction task. For each evaluated edge, the model calculates a prediction score that reflects the likelihood that the edge should exist in the graph [10].

A high prediction score indicates stronger structural support, meaning that the edge fits well within the topology of the citation network. In contrast, a low score may indicate a weak, unstable, or artificial connection.

3.2.7 Stage 5: Stability Computation and Evaluation

Finally, the system computes stability scores across multiple iterations and yearly snapshots. These scores reflect how consistently each citation edge can be reconstructed after being temporarily removed from the graph [1,2].

The evaluation compares original citation edges with injected artificial noise edges. Original edges are expected to receive higher and more consistent stability scores because they are supported by the natural citation network structure. In contrast,

injected noise edges are expected to receive lower scores because they do not necessarily fit the network topology [1,2].

This comparison evaluates whether the system can distinguish between genuine citation links and artificial or structurally unsupported links, using metrics such as AUC-ROC, average stability scores, and comparisons between original and injected edges [10].

3.2.8 Stage 6: Result Saving and Checkpoint Mechanism

The system saves intermediate and final results after each stage. These outputs include edge score files, yearly summaries, iteration histories, and checkpoint files.

This mechanism is important because the experiments are performed on large graphs and may require multiple years and iterations. It allows the experiment to continue from the latest completed stage without restarting.

3.2.9 Testing and Validation

Several validation checks were performed during the research process to ensure that the experimental pipeline works correctly. The yearly graph snapshots were checked to verify that the nodes and citation edges were loaded and filtered properly. The output files were also reviewed after each stage to confirm that edge scores, yearly summaries, and iteration histories were saved correctly.

In addition, the checkpoint mechanism was tested to ensure that interrupted experiments could continue from the latest completed stage without restarting. The results were validated by comparing original citation edges with injected fake/noise edges using stability scores and evaluation metrics such as AUC-ROC. This comparison helped verify that the system behaves as expected and can distinguish between structurally supported and artificial citation links.

3.3 Tools and Technologies

This section describes the main programming languages, libraries, development environment, tools, and datasets used in the implementation of the project.

3.3.1 Programming Language

The project was implemented using **Python** as the main programming language. Python was selected because it supports data processing, graph analysis, machine learning, and scientific computing. It also includes libraries suitable for large datasets, citation networks, graph embeddings, and evaluation metrics.

3.3.2 Development Environment

The implementation and experiments were mainly conducted in **Google Colab**. Google Colab was used because it provides a cloud-based environment that supports Python notebooks, computational resources, and Google Drive integration.

Google Drive was used for storing the dataset, intermediate files, checkpoints, yearly results, and experiment summaries. Since the experiments require multiple iterations and may run for a long time, the use of Google Drive allowed the system to save progress and continue from the last completed stage when needed.

3.3.3 Dataset

The dataset used in this project was based on **OpenAlex**, which is a very large academic database containing millions of scholarly works and citation relationships. Since the full OpenAlex dataset is too large to process directly, we selected only a relevant part of it for our project.

We filtered the original OpenAlex dataset by selecting papers published between 2015 and 2025, then focused only on papers from the Computer Science field and one specific subfield. This allowed us to create a smaller and more focused dataset that is suitable for studying citation behavior and citation trustworthiness.

From the filtered data, we created the main project files: a nodes file representing the papers, and an edges file representing citation links between papers.

In the final graph representation, each paper is treated as a node, and each citation is treated as an edge. This graph was then used to build yearly citation networks, inject artificial noise edges, and evaluate the stability of citation links over time.

3.3.4 Main Libraries and Frameworks

The implementation uses several Python libraries, each supporting a different stage of the research pipeline.

Pandas was used for loading, cleaning, filtering, and processing tabular data, including node files, edge files, yearly summaries, and experiment results.

NumPy was used for numerical operations, vector calculations, and efficient handling of arrays.

NetworkX was used for constructing and manipulating citation graphs. It supports graph creation, edge operations, node analysis, and structural graph computations.

Matplotlib was used for visualizing results, including plots related to stability scores, distributions, and evaluation results.

Scikit-learn was used for evaluation metrics, mainly AUC-ROC, which measures the separation between original citation edges and injected artificial noise edges.

3.3.5 Graph Embedding and Machine Learning Libraries

The project uses two graph embedding approaches: Node2Vec [8] and GraphSAGE [9].

Node2Vec was used to generate node embeddings based on biased random walks over the citation graph. These embeddings capture structural relationships between papers and are used later in the link prediction and reconstruction stage [8].

PyTorch and PyTorch Geometric were used for implementing the GraphSAGE model. GraphSAGE learns node representations by aggregating information from neighboring nodes and node-level features. This makes it suitable for graph-based learning, especially when working with evolving citation networks [9].

3.3.6 Experiment Management and Storage Tools

Several additional tools were used to manage long-running experiments and reduce the risk of losing progress.

Pickle was used to save and load checkpoint files. These checkpoints store the state of the experiment, including removal counts, reconstruction counts, iteration history, and the last completed iteration.

TQDM was used to display progress bars during long-running loops, especially during the evaluation of removed citation edges.

Garbage Collection (gc) was used to manually release memory after large objects were no longer needed.

psutil was used to monitor available RAM during execution, especially before running memory-intensive stages such as Node2Vec training.

shutil was used for managing temporary folders, such as deleting temporary Node2Vec files after an iteration is completed.

3.4 Analytical and Engineering Challenges During Implementation

During the project, several analytical and engineering challenges were encountered. These challenges were mainly related to the large scale of the citation dataset, memory limitations, GPU usage, and long execution time.

3.4.1 Large Dataset Size

One of the main challenges was the large size of the original citation dataset. The full dataset contained many papers and citation links, which made it difficult to process the entire citation network efficiently within the available computational environment.

To address this issue, the project focused on a specific category from the database instead of using the entire dataset. This reduction allowed the system to work with a smaller and more manageable citation graph, while still preserving meaningful citation relationships for the experiment.

3.4.2 RAM Limitations

Another major challenge was RAM consumption. The experimental pipeline requires loading citation data, constructing yearly snapshots, building graph structures, injecting artificial noise edges, removing edges during perturbation, generating embeddings, and computing reconstruction scores [1,2].

These operations require a large amount of memory, especially in later years, where the cumulative citation graph becomes larger. In some cases, the execution reached the available RAM limit, which caused interruptions or failures during the experiment.

To reduce memory usage, the implementation used edge sampling. Instead of evaluating all citation edges in each iteration, only a selected percentage of edges was used as a representative sample. This reduced RAM consumption while still allowing the system to compare original citation edges with injected artificial edges.

3.4.3 GPU Limitations

GPU usage was also a challenge. Although GPU acceleration helped in training GraphSAGE, several stages such as data loading, preprocessing, graph construction, edge sampling, and result saving still depended mainly on CPU and RAM. Therefore, the overall runtime remained high.

3.4.4 Long Execution Time

The experiments were time-consuming because the system processes multiple yearly snapshots and perturbation iterations. Each year requires graph construction, noise injection, embedding generation, link prediction, stability computation, and result saving. This made repeated experiments and parameter tuning slow [1,2].

3.4.5 Lack of Ground-Truth Labels

A central research challenge is the lack of explicit ground-truth labels for manipulated or unreliable citation links. Real citation datasets usually do not indicate whether a specific citation is genuine, weak, or manipulative, although citation manipulation has been discussed in previous studies [3,5,6,11]. This makes it difficult to directly evaluate the model against real confirmed cases of citation manipulation.

To address this limitation, the project used artificial noise injection. Injected edges served as a controlled group simulating citation links without clear structural support, allowing comparison between original citation edges and artificial edges [1,2].

3.4.6 Limitations of the Proposed Solution

The solution provides stability-based indicators, but it does not prove that a citation is fraudulent or manipulative. A low stability score indicates that a citation link has weak structural support in the graph, but additional evidence would be required before making a final academic or ethical judgment.

Another limitation is that the results may depend on the selected dataset, graph construction method, embedding parameters, sampling strategy, and threshold values. In addition, converting the citation graph into an undirected graph helps focus on structural relationships, but it ignores the original citation direction.

Despite these limitations, the framework provides a useful approach for analyzing citation stability and identifying links that may require further investigation.

4. Results, Conclusions, and Project Evaluation

This section presents the main experimental results of the framework and evaluates whether the project achieved its research goals. The analysis compares original citation edges with injected noise edges using stability scores, AUC values, coefficient of variation (CV), and yearly visualizations [1,2].

4.1 Results Visualization

The experimental results were visualized to analyze and compare the behavior of original citation edges and injected noise edges over time.

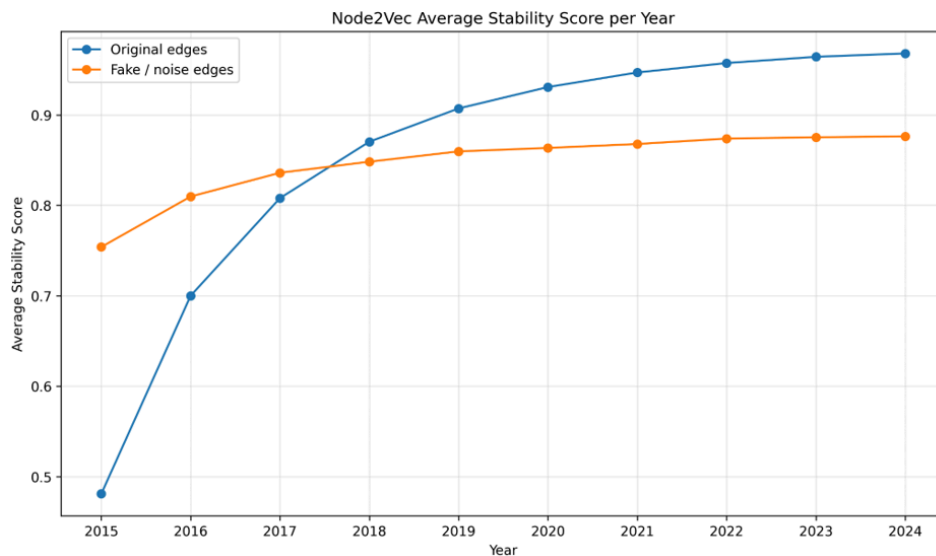


Figure 3. Node2Vec average stability score per year.

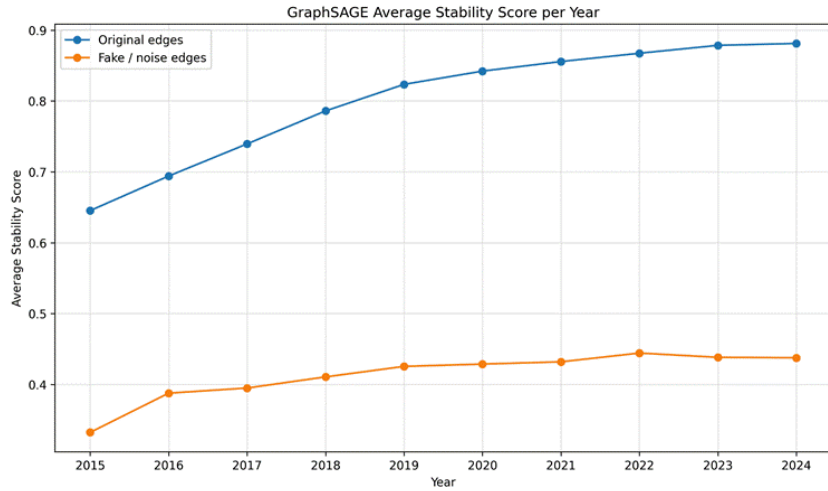


Figure 4. GraphSAGE average stability score per year.

Figures 3 and 4 show that the separation between original citation edge and injected noise edges becomes clearer over time. GraphSAGE demonstrates a stronger and more consistent separation than Node2Vec, indicating superior performance in distinguishing between edge types .

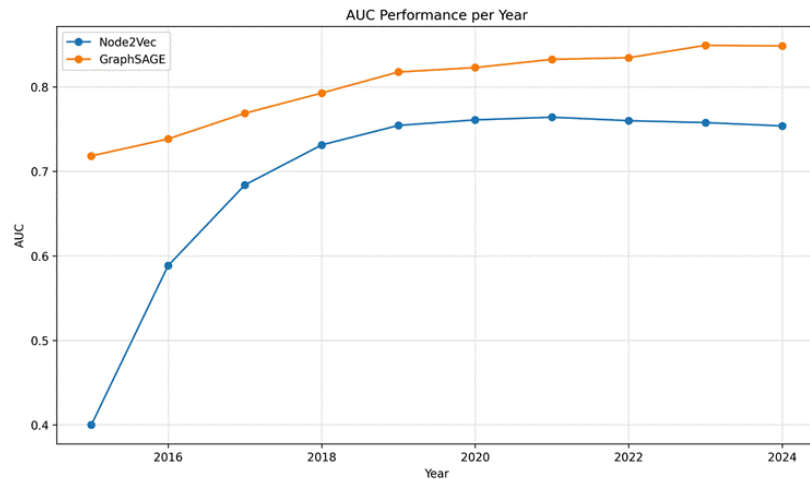


Figure 5. AUC performance per year for Node2Vec and GraphSAGE.

Figure 5 shows that GraphSAGE achieves higher and more stable AUC values than Node2Vec. Node2Vec improves as the graph grows, but GraphSAGE provides better separation between original and injected citation edges.

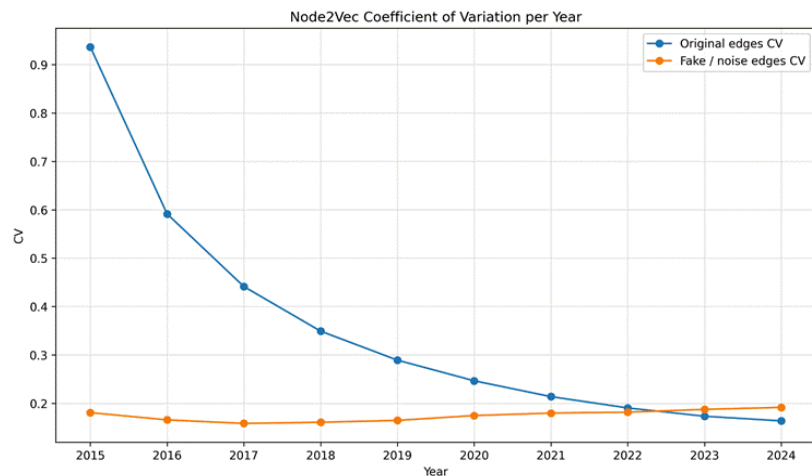


Figure 6. Node2Vec coefficient of variation per year.

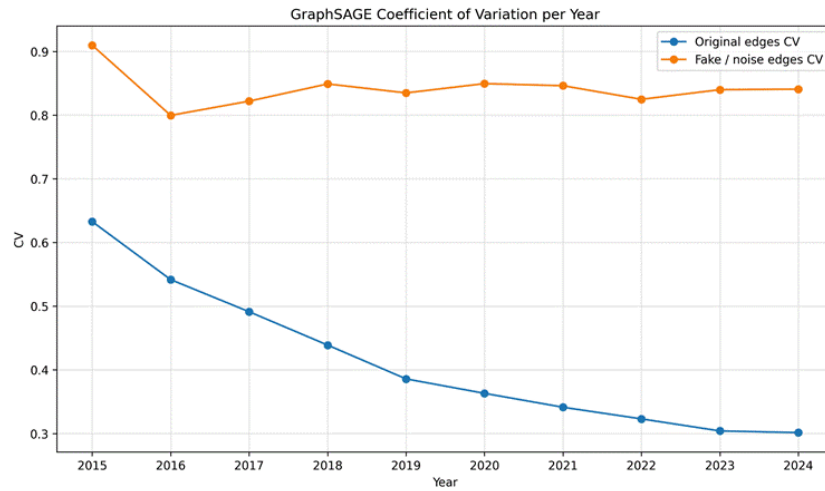


Figure 7. GraphSAGE coefficient of variation per year.

Figures 6 and 7 show the CV results for Node2Vec and GraphSAGE. In both models, the CV of original citation edges decreases over time, indicating that original citations become more stable as the graph grows. GraphSAGE shows a clearer separation between original citation edges and injected noise edges, while Node2Vec becomes more stable mainly in later years.

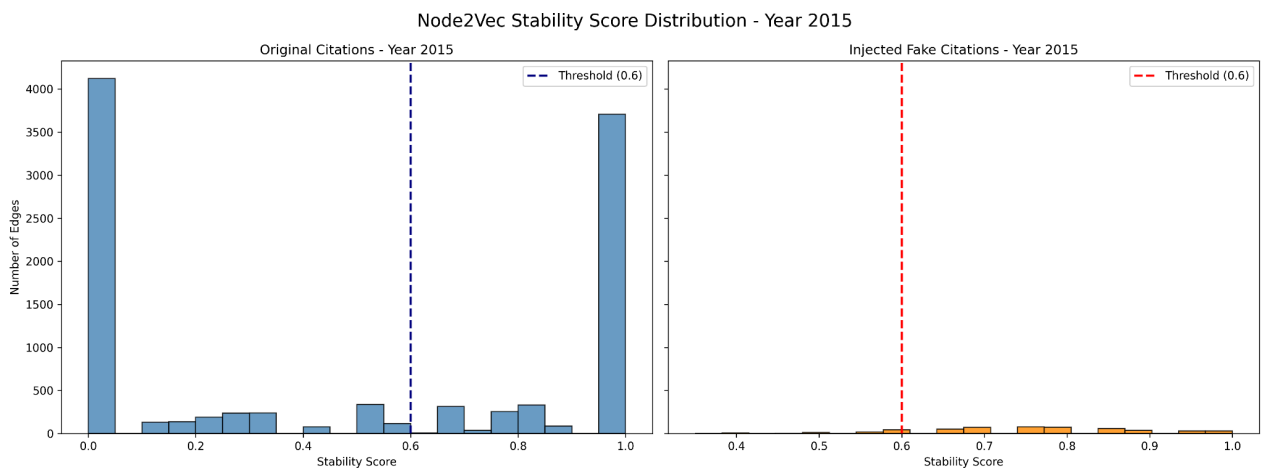


Figure 8. Node2Vec stability score distribution in 2015.

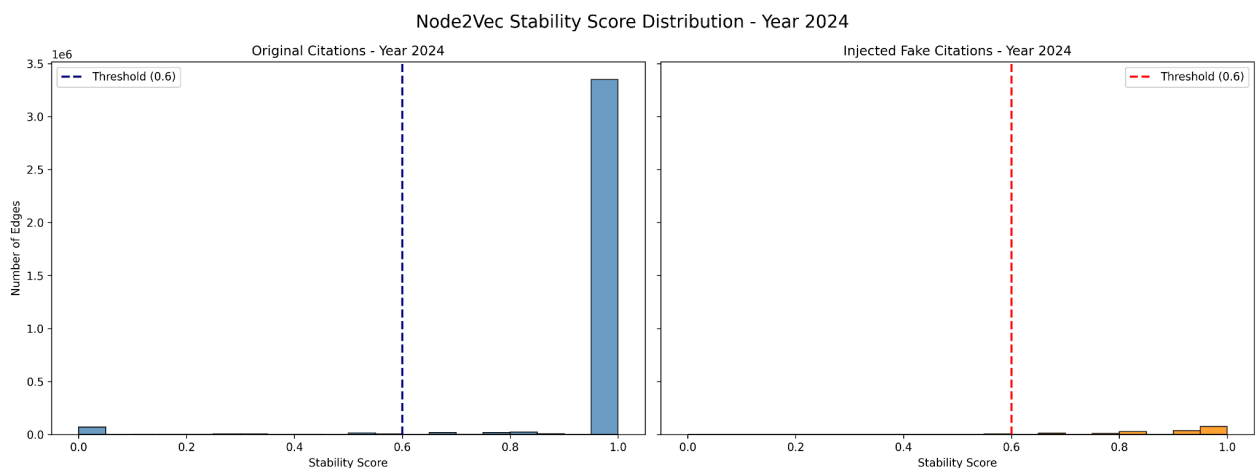


Figure 9. Node2Vec stability score distribution in 2024.

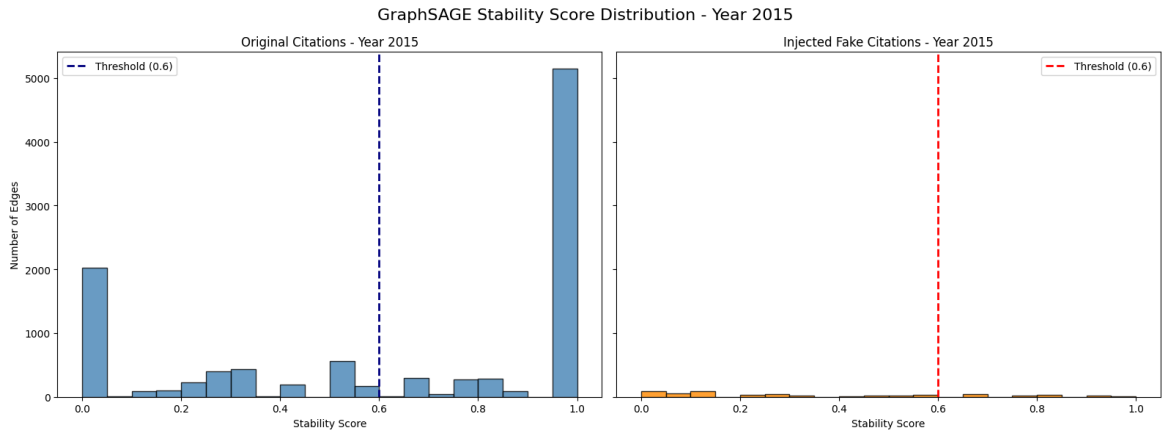


Figure 10. GraphSAGE stability score distribution in 2015.

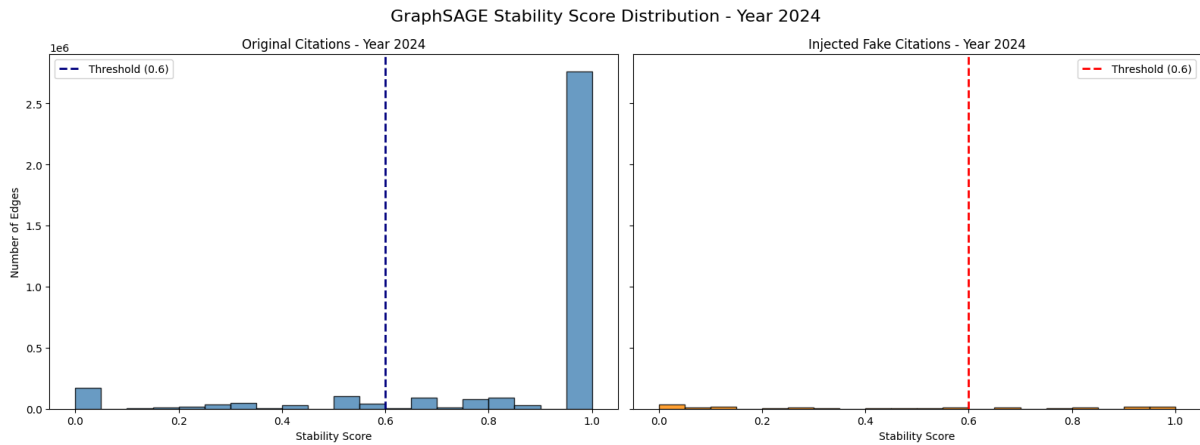


Figure 11. GraphSAGE stability score distribution in 2024.

Figures 8–11 compare the stability score distributions of original citation edges and injected citation edges in early and late snapshots. The results show that, in 2024, original citation edges are more concentrated around high stability scores than in 2015, indicating stronger structural support in larger graphs. GraphSAGE shows a clearer separation between original and injected edges, while Node2Vec also improves as the graph becomes richer over time.

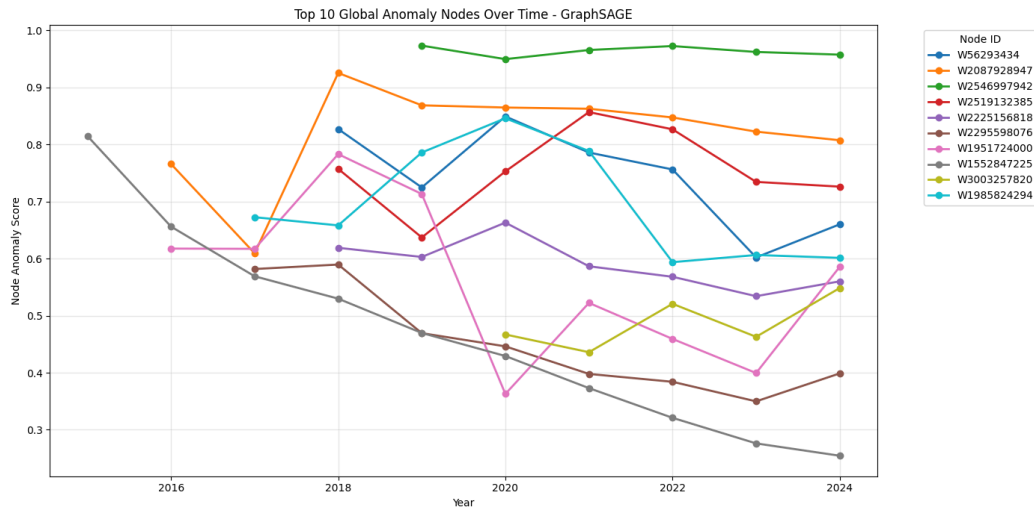


Figure 12. Top 10 global anomalous nodes over time according to GraphSAGE.

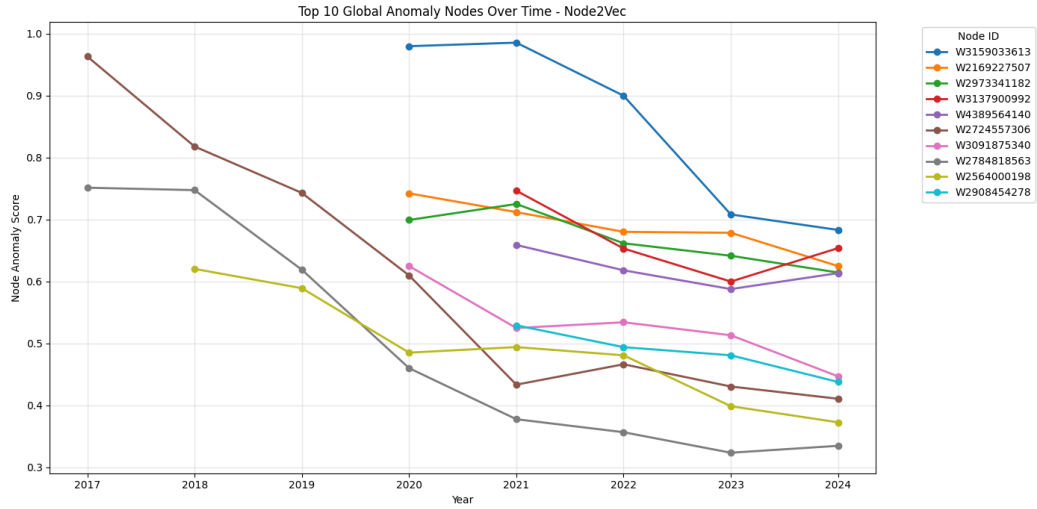


Figure 13. Top 10 global anomalous nodes over time according to Node2Vec.

Figures 12 and 13 present the top 10 anomalous nodes over time for Node2Vec and GraphSAGE. The results show that some papers are connected to less stable citation edges, meaning that suspicious behavior can also appear at the paper level, not only at the citation-link level. In Node2Vec, some nodes become less anomalous as the graph grows, while in GraphSAGE several nodes remain highly anomalous across multiple years. This supports stability-based analysis for identifying papers whose citation behavior may require further investigation.

4.2 Conclusions

The results show that the project achieved its main goal: evaluating citation trustworthiness using graph-based stability analysis [1,2]. Both Node2Vec [8] and GraphSAGE [9] were used to compare original citation edges with injected noise edges, but GraphSAGE produced more consistent and robust results.

The average stability score, AUC, CV, distribution graphs, and node-level anomaly analysis all support the conclusion that original citation links are generally more structurally stable than injected artificial links.

The results also show that the analysis improves as the citation graph grows. In later years, the network contains more papers and citation links, giving models more structural information and making separation between original citation edges and injected noise edges more pronounced [4].

4.3 Project Goals Evaluation

The project met the main Phase B goals. We built yearly cumulative citation snapshots, injected artificial noise edges, applied Node2Vec [8] and GraphSAGE [9], computed stability scores, and compared original citation edges with injected edges.

We also evaluated the results using average stability score, AUC, coefficient of variation, stability score distributions, and top 10 anomalous nodes.

Therefore, the implemented framework provides a complete experimental pipeline for analyzing citation link stability over time. The system does not prove that a citation is fraudulent, but it provides interpretable structural indicators that can help identify citation links or papers that may require further investigation [1,2].

4.4 Decision Making and Handling Challenges

Several decisions were made to handle the research and engineering challenges. Since there were no real ground-truth labels for suspicious citations, we used artificial noise edge injection as a controlled comparison group [1,2].

Since the citation network was large and grew over time, we used yearly cumulative snapshots, edge sampling, checkpoint files, and SQLite progress storage to reduce memory usage and avoid losing progress.

We used two graph-based methods. Node2Vec [8] was selected as a graph embedding baseline, while GraphSAGE [9] was selected because it uses node features and neighborhood information. This allowed us to compare methods and evaluate which better separates original citation edges from injected noise edges.

4.5 Lessons Learned

One main lesson is that large evolving graphs require careful planning of memory usage, runtime, and output storage. Checkpointing was essential because the experiments were long and sometimes interrupted by Colab limitations.

We also learned that evaluation criteria affect the final results. For example, in node-level anomaly analysis, selecting nodes without enough structural evidence may lead to misleading results. Therefore, filtering, weighting, and result validation are important parts of the analysis.

If we repeated the project, we would define the output structure, evaluation metrics, anomaly selection criteria, and visualization plan earlier, to improve consistency and reduce later adjustments.

4.6 Future Work

Future work can extend this project in several directions. The analysis can be tested on additional academic fields and larger OpenAlex subsets. The citation graph can also be analyzed as a directed graph to preserve citation direction. In addition, textual information from papers, such as titles and abstracts, can be combined with graph structure. Finally, real labeled cases of suspicious citations, if available, can be used to validate the method more directly [3,5,6,11].

References

1. **Avros, R.**; Keshet, S.; Toledano Kitai, D.; Vexler, E.; Volkovich, Z. Detecting Pseudo-Manipulated Citations in Scientific Literature through Perturbations of the Citation Graph. *Mathematics*, 11(18), 3820, 2023.
2. **Avros, R.**; Toledano Kitai, D.; Volkovich, Z. Citation Steadiness Analysis with GraphSAGE Approach. *Mathematics*, 11(18), 3820, 2023.
3. **Biagioli, M.** Watch out for cheats in citation game. *Nature*, 535(7611), 201, 2016.
4. **Ferreira, C. H.**, & Murai, F. On the stability of citation networks. *Physica A: Statistical Mechanics and its Applications*, 610, 128399, 2023.
5. **Fister Jr, I.**; Fister, I.; Perc, M. Toward the discovery of citation cartels in citation networks. *Frontiers in Physics*, 4, 49, 2016.
6. **Fong, E. A.**, & Wilhite, A. W. Authorship and citation manipulation in academic research. *PLOS ONE*, 12(12), e0187394, 2017.
7. **Garfield, E.** Citation indexes for science: A new dimension in documentation through association of ideas. *Science*, 122(3159), 108–111, 1955.
8. **Grover, A.**; Leskovec, J. Node2vec: Scalable Feature Learning for Networks. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD '16)*, San Francisco, CA, USA, August 13–17, 2016; pp. 855–864.
9. **Hamilton, W.**; Ying, Z.; Leskovec, J. Inductive representation learning on large graphs. In *Advances in Neural Information Processing Systems (NIPS)*, Vol. 30, 2017.
10. **I, B. K.**; Mathi, A. R. P.; Sett, N. Evaluating link prediction: New perspectives and recommendations. *arXiv preprint arXiv:2502.12777*, 2025.
11. **Wilhite, A. W.**; Fong, E. A. Coercive citation in academic publishing. *Science*, 335(6068), 542–543, 2012.

Appendices:

A. User Guide

- **Purpose**

This user guide provides instructions for running the experimental pipeline developed in this project. The guide describes the required environment, input files, folder structure, execution steps, and output files needed to reproduce the citation trustworthiness analysis.

The purpose of the project is to analyze the structural stability of citation links in evolving scholarly networks and examine whether citation edges are structurally supported by the underlying graph topology.

- **System Overview**

The system is a research-oriented computational framework for analyzing citation trustworthiness in evolving citation networks. It is not intended to be a commercial software product, but rather an experimental pipeline for studying whether citation links are structurally supported by the citation graph.

The pipeline follows several main stages:

1. Load citation data from CSV files.
2. Build yearly cumulative citation graph snapshots.
3. Construct an undirected citation graph for each year.
4. Inject artificial noise edges.
5. Temporarily remove selected edges from the graph.
6. Generate node embeddings using Node2Vec and GraphSAGE.
7. Perform link prediction using embedding similarity.
8. Compute stability scores for original edges and injected noise edges.
9. Save iteration history, yearly summaries, checkpoints, SQLite database files, and CV analysis results.

For each yearly snapshot, the citation network is treated as a static graph. The temporal behavior is captured by repeating the same structural stability analysis across cumulative yearly snapshots.

- **Requirements**

The project requires the following environment and tools:

- Google Colab.
- Python via Google Colab.
- Google Drive account.

- Recommended: High-RAM runtime for all experiments.
- Recommended: GPU runtime mainly for GraphSAGE experiments.
- Sufficient Google Drive storage for datasets, snapshots, checkpoints, SQLite database, and result files.

The experiments may require a long execution time, especially for later years such as 2023 and 2024, because the cumulative citation graphs become significantly larger over time.

● Repository Location

The project repository is available at:

<https://github.com/Manal2308/Final-Project.git>

● Files Provided

The main files used in the project are:

- `Final_project.ipynb`: the main notebook containing the full experimental pipeline.
- `nodes.csv`: contains paper information and metadata.
- `edges.csv`: contains citation relationships between papers.

The dataset was built from OpenAlex data. The data collection process retrieves scholarly works and constructs `nodes.csv` and `edges.csv` from the collected citation relationships.

● Setup Instructions

1. Open Google Colab.
2. Unzip `openalex_cs_dataset.zip`.
3. Upload the extracted dataset folder to your Google Drive.
4. Open the main notebook: `Final_project.ipynb`
5. Mount Google Drive when prompted by the notebook.
6. Update the file paths in the notebook if needed (according to your folder structure).
7. Run the notebook cells in order.

● Running the Pipeline

To run the experiment, execute the notebook cells from top to bottom.

The main execution flow is:

1. Import libraries.
2. Define global parameters.
3. Load `nodes.csv` and `edges.csv`.

4. Build or load yearly cumulative snapshots.
5. Prepare the citation graph for the selected year.
6. Inject artificial noise edges.
7. Run Node2Vec or GraphSAGE embedding-based stability analysis.
8. Save yearly results and iteration history files.
9. Compute yearly summaries and CV analysis.

The system supports resuming long-running experiments. GraphSAGE uses checkpoint files, while Node2Vec uses a SQLite database to store progress and edge-level counts.

- **Google Drive Folder Structure**

```

/content/drive/MyDrive/Final_project/
├──
├── snapshots/
│   ├── nodes_snapshot_2015.csv
│   ├── edges_snapshot_2015.csv
│   ├── ...
│   ├── nodes_snapshot_2024.csv
│   └── edges_snapshot_2024.csv
├──
├── graphsage_checkpoints/
│   ├── graphsage_stability_checkpoint_2015.pkl
│   ├── ...
│   └── graphsage_stability_checkpoint_2024.pkl
├──
├── graphsage_results/
│   ├── graphsage_iteration_history_year_2015.csv
│   ├── ...
│   ├── graphsage_iteration_history_year_2024.csv
│   ├── year_result_2015.csv
│   ├── ...
│   ├── year_result_2024.csv
│   ├── all_years_summary.csv
│   ├── graphsage_detailed_cv_per_year.csv
│   └── graphsage_detailed_cv_global.csv
├──
├── node2vec_results/
│   ├── node2vec_stability_results.db
│   ├── node2vec_iteration_history_year_2015.csv
│   ├── ...
│   ├── node2vec_iteration_history_year_2024.csv
│   ├── node2vec_year_result_2015.csv
│   ├── ...
│   ├── node2vec_year_result_2024.csv
│   ├── node2vec_all_years_summary.csv
│   ├── node2vec_detailed_cv_per_year_from_db.csv
│   └── node2vec_detailed_cv_global_from_db.csv
└──

```

- **Folder Description**

After running the full pipeline, the following directories are created:

- **snapshots**
Stores yearly cumulative graph snapshots. Each year has a node file and an edge file.
 - **graphsage_checkpoints**
Stores GraphSAGE checkpoint files. These files allow the experiment to resume from the last completed iteration.
 - **graphsage_results**
Stores GraphSAGE output files, including iteration history, yearly results, all-years summary, and CV analysis results.
 - **node2vec_results**
Stores Node2Vec output files, including the SQLite database, yearly result files, iteration history files, all-years summary, and CV analysis files.
- **Main Output Files**
The main output files include:
 - **nodes_snapshot_YEAR.csv, edges_snapshot_YEAR.csv:**
Store the cumulative citation graph snapshot for each year.
 - **node2vec_stability_results.db:**
Stores Node2Vec edge-level counts and progress using SQLite for original and injected edges.
 - **node2vec_iteration_history_year_YEAR.csv:**
Stores compact Node2Vec iteration-level information.
 - **node2vec_year_result_YEAR.csv:**
Stores the Node2Vec yearly summary.
 - **node2vec_all_years_summary.csv:**
Stores the Node2Vec summary across all evaluated years.
 - **node2vec_detailed_cv_per_year_from_db.csv:**
Stores per-year Node2Vec CV analysis results.
 - **node2vec_detailed_cv_global_from_db.csv:**
Stores global Node2Vec CV analysis results.
 - **graphsage_iteration_history_year_YEAR.csv:**
Stores compact GraphSAGE iteration-level information.
 - **year_result_YEAR.csv:**
Stores the GraphSAGE yearly summary.
 - **all_years_summary.csv:**
Stores the GraphSAGE summary across all evaluated years.
 - **graphsage_cv_results.csv:**
Stores the GraphSAGE CV analysis results.

The system includes checkpoint and resume mechanisms to support long-running experiments, especially for large yearly snapshots such as 2023 and 2024.

- **Notes and Recommendations**

- Use a High-RAM runtime when possible.
- For large years such as 2023 and 2024, it is recommended to run one or a few iterations at a time.

- **Expected Runtime Considerations**

The runtime depends on the graph size, year, number of iterations, and embedding method. Early years are smaller and run faster, while later cumulative snapshots require significantly more memory and time.

Node2Vec may become computationally expensive for large graphs because it generates random walks and trains embeddings for many nodes.

- **Final Notes**

This user guide describes how to run the project pipeline and manage its input files, output files, checkpoints, SQLite database files, and results.

The system is intended for research users who want to analyze citation stability, compare original citation edges with injected noise edges , and evaluate citation trustworthiness using graph-based stability analysis methods.

B. Maintenance Guide

- **Project Overview**

This project analyzes the trustworthiness of citation links in evolving scholarly citation networks using data from the **OpenAlex** database. The goal is to identify citation links that may be weak, unstable, or suspicious based on the graph structure.

- **Key Tools**

- Python (Google Colab)
- OpenAlex Database
- NetworkX (Citation Graph Construction)
- Node2Vec (Graph Embedding Model)
- GraphSAGE (Graph Neural Network Model)
- PyTorch
- PyTorch Geometric
- Pandas and NumPy
- Scikit-learn
- Matplotlib
- SQLite for Node2Vec progress storage
- Pickle checkpoints for GraphSAGE

- **Environment Setup**

This project is designed to run in Google Colab with Python 3 and GPU support enabled.

Required Libraries:

```
!pip install torch torch_geometric  
  
!pip install node2vec
```

Directory Structure:

```
/content/drive/MyDrive/
|
├─ Final_project/          # Main project folder
|   |
|   ├─ snapshots/          # Yearly citation graph snapshots
|   |
|   ├─ graphsage_checkpoints/ # Saved GraphSAGE checkpoints
|   |
|   ├─ graphsage_results/   # GraphSAGE results and edge scores
|   |
|   └─ node2vec_results/    # Node2Vec results and edge scores
└─ openalex_cs_dataset/
    |
    ├─ nodes.csv           # Papers/nodes information
    |
    └─ edges.csv           # Citation links between papers
```

The `Final_project` directory stores the generated snapshots, checkpoints, SQLite database, and result files.

The `openalex_cs_dataset` directory stores the original input dataset files.

- **Data Preparation & Preprocessing:**

The dataset is based on the OpenAlex database and focuses on publications from the Computer Science field.

The dataset consists of two main files:

- **Nodes File:**
Contains information about the papers in the network.
Each paper is represented as a node.
- **Edges File:**
Contains citation relationships between papers.
Each citation is represented as an edge between two nodes.

```
/content/drive/MyDrive/openalex_cs_dataset/
|
├─ nodes.csv      # Papers information
|
└─ edges.csv      # Citation links between papers
```

Preprocessing Steps:

The preprocessing stage prepares the OpenAlex data for graph-based analysis.

The pipeline performs the following steps:

- Mounts Google Drive and loads the OpenAlex dataset files.
 - Reads `nodes.csv` and `edges.csv`.
 - Removes isolated nodes if the column `is_isolated_in_dataset` exists.
 - Filters papers according to `publication_year`.
 - Builds cumulative yearly snapshots from **2015 to 2024**.
 - Keeps only valid citation edges where both source and target papers exist in the selected snapshot.
 - Converts the citation data into an undirected citation graph using NetworkX.
 - Adds injected noise edges to simulate fake citation links.
 - Saves yearly snapshot files for later use.
- **Scripts:**

Loading the dataset:

```
all_nodes_df = pd.read_csv(nodes_path)
all_edges_df = pd.read_csv(edges_path)
```

Building yearly snapshots:

```
def build_snapshot_for_year(all_nodes_df, all_edges_df,
    current_year, snapshot_mode="cumulative"):
```

Preparing the graph for each year:

```
def prepare_graph_for_year(nodes_df, edges_df,
    current_year):
```

These functions perform the main preprocessing operations:

- Read the OpenAlex nodes and edges data.
 - Filter nodes by publication year.
 - Remove invalid citation links.
 - Build the citation graph.
 - Add injected noise edges for comparison.
 - Prepare the graph data for Node2Vec and GraphSAGE.
- **GraphSAGE Preparation:**

The project uses **GraphSAGE** as a graph neural network model for citation link stability analysis.

GraphSAGE Processing Steps:

The GraphSAGE pipeline performs the following steps:

- Loads the yearly nodes and edges snapshot.
- Builds the citation graph for the selected year.
- Adds artificial noise edges to represent injected citations.
- Converts the graph into PyTorch Geometric format.
- Maps paper IDs into numeric node indices.
- Builds the edge_index tensor.
- Selects and normalizes numeric node features.
- Trains a GraphSAGE model for link prediction.
- Removes sampled original edges during each iteration.
- Checks whether the removed edges are reconstructed.
- Calculates stability scores for original and injected noise edges.
- Saves checkpoints and yearly result files.

Script:

This function prepares the yearly graph for GraphSAGE training.

```
def prepare_graphsage_data_for_year(Graph_C, nodes_df):
```

This function runs the GraphSAGE stability analysis for one year.

```
def run_graphsage_stability_for_year(year, all_edges,
original_edges, noise_edges, node_to_idx, data_full,
target_iterations):
```

Output:

The GraphSAGE checkpoints are saved in:

/content/drive/MyDrive/Final_project/graphsage_checkpoints/

The GraphSAGE result files are saved in:

/content/drive/MyDrive/Final_project/graphsage_results/

The outputs include stability scores, reconstruction counts, AUC values, and yearly GraphSAGE analysis results.

- **Node2Vec Stability Analysis:**

The project uses Node2Vec to analyze the stability of citation links in the OpenAlex citation graph.

Node2Vec Processing Steps:

The Node2Vec pipeline performs the following steps:

- Loads the yearly citation snapshot.
- Builds the citation graph for the selected year.
- Adds artificial noise edges for comparison.
- Uses a local SQLite DB file to save progress and checkpoint data.
- Removes sampled original citation edges during each iteration.
- Trains a Node2Vec model on the modified graph.
- Generates embeddings for papers in the graph.
- Calculates similarity scores between node pairs.
- Compares original edges with injected noise edges.
- Computes stability scores for citation links.
- Saves the yearly Node2Vec results.

Script:

This function runs the Node2Vec stability experiment for one selected year.

```
def run_node2vec_stability_for_year_db(year, graph_full,
original_edges, noise_edges, target_iterations,
db_file=NODE2VEC_DB_FILE):
```

Local DB File:

A local SQLite database file is used during execution to store intermediate Node2Vec results:

/content/node2vec_stability_results.db

It stores information such as:

- Completed iterations.

- Removed original edges.
- Fake/noise edges.
- Reconstruction counts.
- Stability scores.
- Checkpoint progress.

After completed iterations, the local database is copied back to Google Drive:

```
/content/drive/MyDrive/Final_project/node2vec_results/  
node2vec_stability_results.db
```

Output:

The Node2Vec results are saved in:

```
/content/drive/MyDrive/Final_project/node2vec_results
```

This folder contains the SQLite database, yearly result files, iteration history files, all-years summary files, CV analysis files, and final Node2Vec evaluation results.

• **Comparison & Evaluation:**

The comparison and evaluation stage is used to analyze the behavior of original citation edges compared to injected noise edges. This stage is important for validating whether the implemented models can distinguish between structurally supported citation links and artificial links.

The evaluation is performed separately for Node2Vec and GraphSAGE. For each year, the system calculates stability scores for original and injected edges. These scores are then used to generate yearly summaries, AUC-ROC values, coefficient of variation results, stability score distributions, and anomaly analysis outputs.

The main evaluation outputs include:

- Average stability score of original edges.
- Average stability score of injected noise edges.
- AUC-ROC score for separating original and injected edges.
- Coefficient of Variation (CV) for stability score consistency.
- Stability score distribution plots.
- Top anomalous nodes based on node-level stability.

The expected behavior is that original citation edges should usually receive higher and more consistent stability scores, while injected noise edges should

receive lower scores. A higher AUC-ROC value indicates better separation between original and injected edges.

The generated results are saved in the corresponding result folders:

```
/content/drive/MyDrive/Final_project/node2vec_results/  
/content/drive/MyDrive/Final_project/graphsage_results/  
/content/drive/MyDrive/Final_project/visualization_results/
```

When maintaining or extending the project, the developer should verify that the result files are generated for all evaluated years, that the original and injected edge groups are both included, and that the summary files match the plotted graphs. If new experiments are added, the same evaluation structure must be preserved to allow comparison across models and years.